

Grado en Ingeniería del Software
Doble Grado en Matemática Computacional e Ingeniería del Software
Doble Grado en Física Computacional e Ingeniería del Software



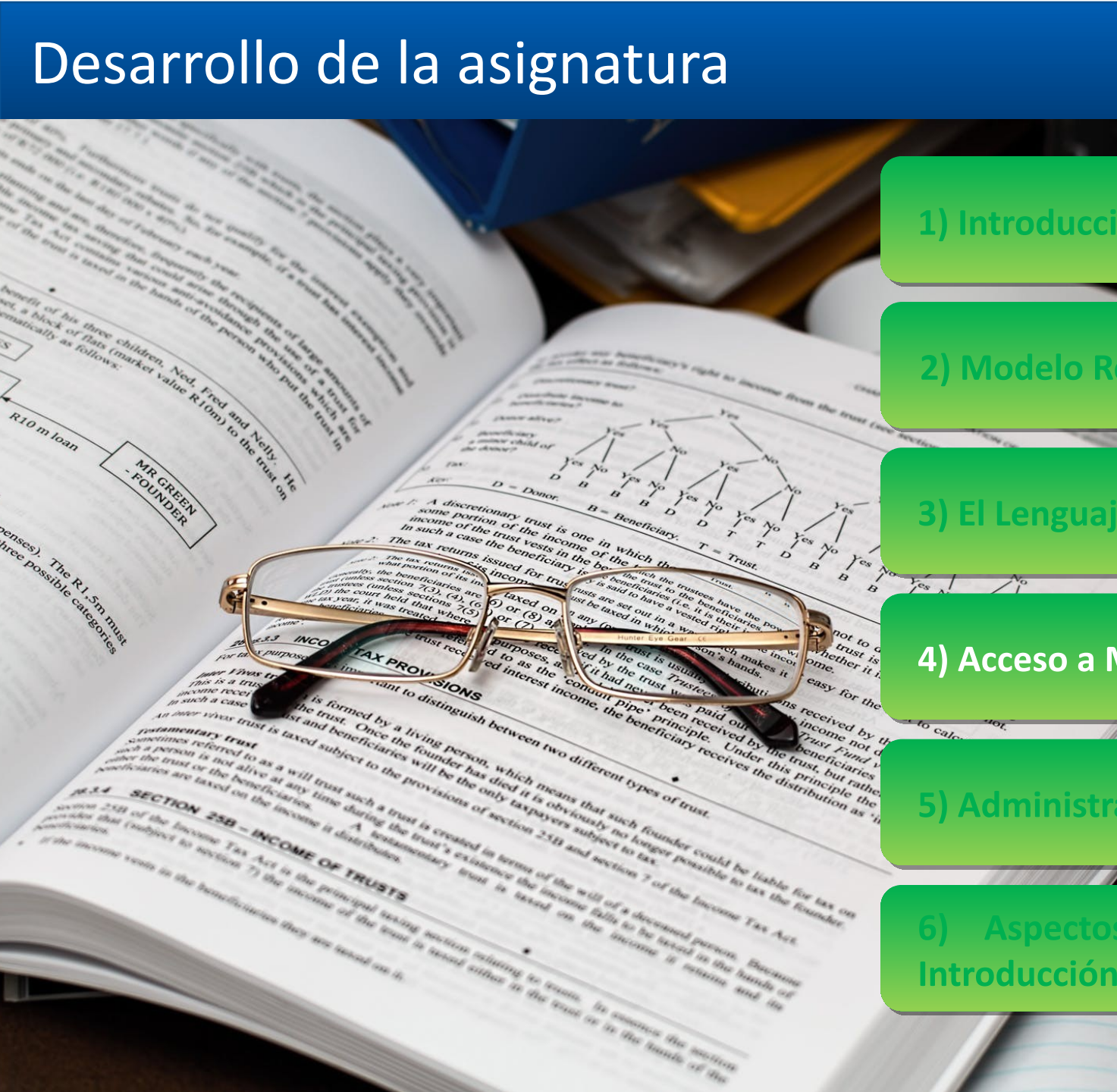
Bases de Datos

Tema 4

Rafael Socas Gutiérrez
Amador Maho Etohá
Ana Isabel Sánchez González

Enero 2025





1) Introducción a las Bases de Datos

2) Modelo Relacional y Algebra Relacional

3) El Lenguaje SQL y SQL Avanzado

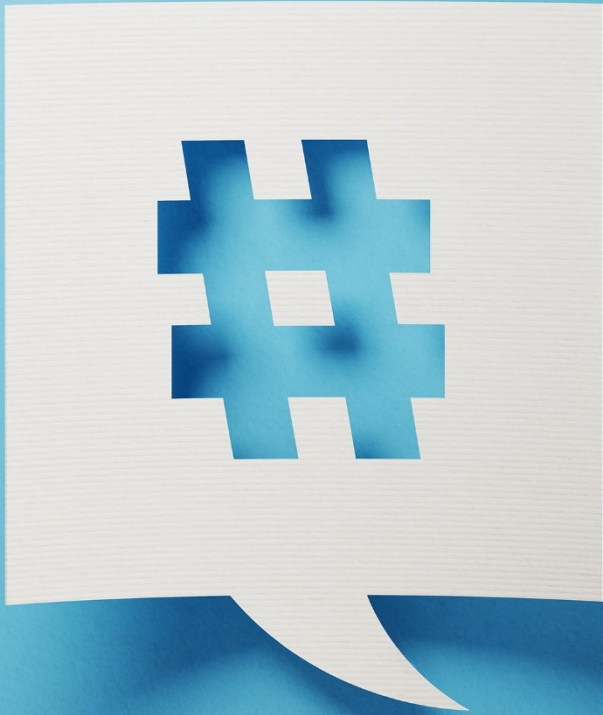
4) Acceso a MySQL, Almacenamiento e Indexado

5) Administración de Bases de Datos en MySQL

6) Aspectos Avanzados sobre BBDD Relacionales e Introducción a las BBDD No Relacionales: MongoDB

4

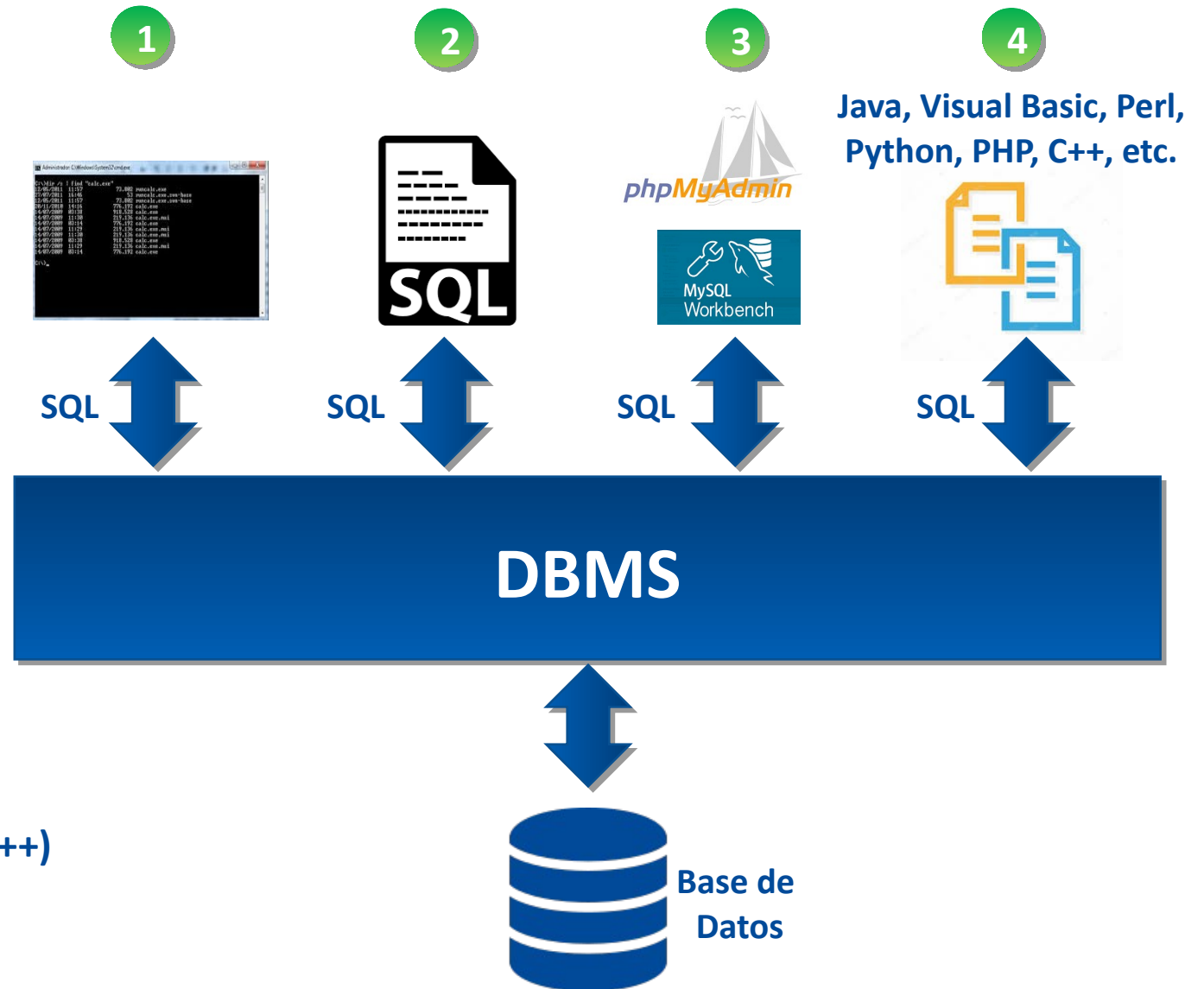
Acceso a MySQL, Almacenamiento e Indexado



1. *Formas de Acceso a BBDD*
2. *Acceso por Consola*
3. *Interacción mediante Scripts*
4. *Interacción a través de Lenguajes de Programación: Python y PHP*
5. *Tipos de Almacenamiento*
6. *Jerarquías*
7. *Indexado*

Formas de interactuar con la BBDD

- 1** Comandos SQL en la consola del DBMS
- 2** Mediante Scripts
- 3** Herramientas de Gestión Workbench/phpMyAdmin
- 4** A través de otro programa (Java, Visual Basic, Perl, Python, PHP, C++)



Interacción con la BBDD: Comandos SQL (1/2)

```
Símbolo del sistema - mysql -u root -p
Microsoft Windows [Versión 10.0.18362.418]
(c) 2019 Microsoft Corporation. Todos los derechos reservados.

C:\Users\Rafael>mysql -u root -p
Enter password: *****

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.7.27-log MySQL Community Server (GPL)

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| academia_idiomas |
| employees |
| mysql |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
8 rows in set (0.00 sec)
```

1

Comandos SQL en la consola del DBMS

Accedemos al DBMS

Mostramos las BBDD que hay en el DBMS

Interacción con la BBDD: Comandos SQL (2/2)

1 Comandos SQL en la consola del DBMS

Seleccionamos la BBDD *academia_idiomas* y mostramos sus tablas

Consultamos todos los Registros de la tabla *clientes*

```
Símbolo del sistema - mysql -u root -p

mysql> use academia_idiomas
Database changed
mysql> show tables;

+-----+
| Tables_in_academia_idiomas |
+-----+
| clientes                    |
| matriculas                  |
| programa                    |
| suscripcion                  |
+-----+
4 rows in set (0.00 sec)

mysql> select * from clientes;

+-----+-----+
| cliente_id | nombre_cliente |
+-----+-----+
| 1          | Pedro          |
| 2          | Aurelia        |
| 3          | Federico       |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

3. Interacción mediante Scripts

Interacción con la BBDD: Scripts



Mediante Scripts

Script script_example.sql

```
script_example: Bloc de notas
Archivo Edición Formato Ver Ayuda
select *
from clientes
```

Ejecutamos el Script

```
Símbolo del sistema
c:\Temp>dir
El volumen de la unidad C es Windows
El número de serie del volumen es: 18C9-DE12

Directorio de c:\Temp

31/10/2019  00:10    <DIR>          .
31/10/2019  00:10    <DIR>          ..
31/10/2019  00:13                24 script_example.sql
                1 archivos             24 bytes
                2 dirs  673.061.302.272 bytes libres

c:\Temp>mysql -u root -p academia_idiomas < script_example.sql
Enter password: *****

cliente_id  nombre_cliente
1          Pedro
2          Aurelia
3          Federico

c:\Temp>
```


3. Interacción mediante Scripts

Interacción con la BBDD: Herramientas de Gestión

Query

3

Herramientas de Gestión Workbench/phpMyAdmin

The image displays two screenshots of database management tools. The left screenshot shows MySQL Workbench with a query window containing the SQL statement `select * from clientes;`. Below the query window, a 'Result Grid' shows the results of the query. The right screenshot shows phpMyAdmin with the same query executed, displaying a confirmation message and the results in a table format.

MySQL Workbench Query:

```
1 • select * from clientes;
```

MySQL Workbench Result Grid:

cliente_id	nombre_cliente
1	Pedro
2	Aurelia
3	Federico
NULL	NULL

phpMyAdmin Query:

```
SELECT * FROM clientes
```

phpMyAdmin Results Table:

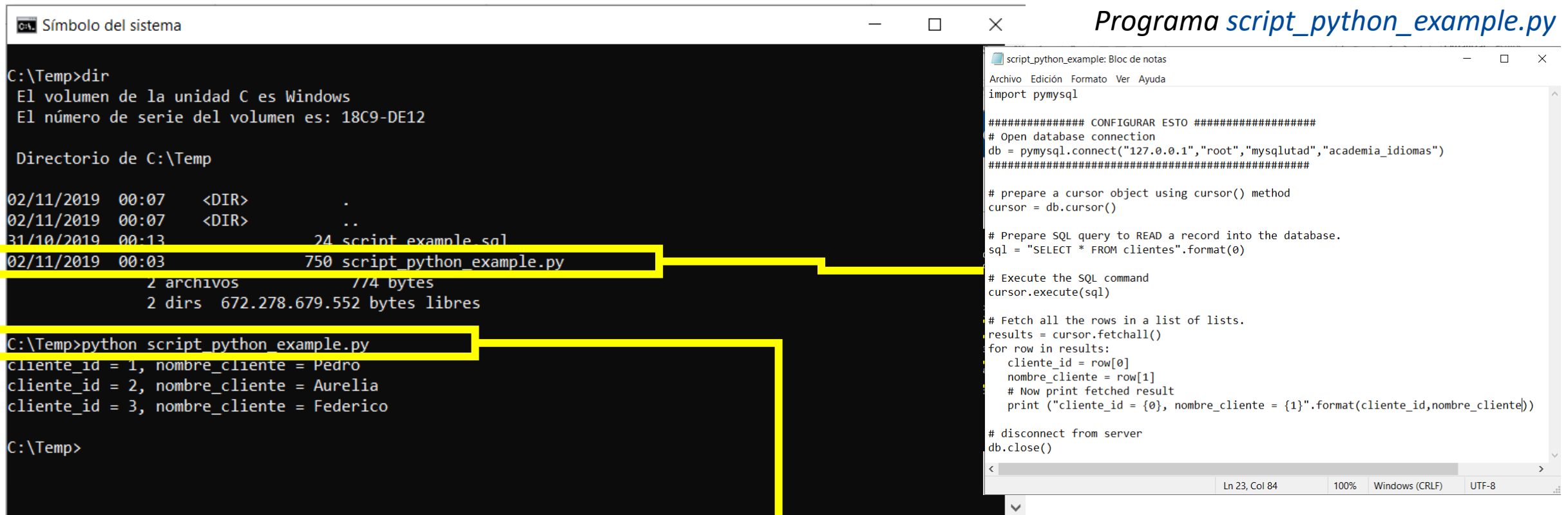
cliente_id	nombre_cliente
1	Pedro
2	Aurelia
3	Federico

Resultado

Ejecución Script SQL en Python

4

A través de otro programa
(Perl, Python, PHP, C++)



The screenshot shows a Windows command prompt window titled 'Símbolo del sistema' and a Notepad window titled 'Programa script_python_example.py'.

In the command prompt, the user runs `dir` in the `C:\Temp` directory. The output shows a list of files, including `script_example.sql` and `script_python_example.py`. The file `script_python_example.py` is highlighted with a yellow box.

The user then runs `python script_python_example.py`. The output shows the results of the SQL query: `cliente_id = 1, nombre_cliente = Pedro`, `cliente_id = 2, nombre_cliente = Aurelia`, and `cliente_id = 3, nombre_cliente = Federico`. This command is also highlighted with a yellow box.

The Notepad window shows the Python script `script_python_example.py`. The script imports `pymysql`, connects to a MySQL database, executes a SQL query to select all records from the `clientes` table, and prints the results. The script is also highlighted with a yellow box.

Ejecutamos el Programa

Integración SQL en Python

En Python, el acceso a bases de datos se encuentra definido a modo de estándar en las especificaciones de DB-API, que [puedes leer en la PEP 249](#). Esto, significa que independientemente de la base de datos que utilicemos, los métodos y procesos de conexión, lectura y escritura de datos, desde Python, siempre serán los mismos, más allá del conector.

En nuestro caso particular, utilizaremos MySQL, para lo cual, vamos a trabajar con el módulo `MySQLdb`.

A diferencia de los módulos de la librería estándar de Python, `MySQLdb` debe ser instalado manualmente. Para ello, ejecutaremos el siguiente comando:

```
$ python3 --versión # Conocer la versión de Python 2,3 etc.  
$ apt update  
$ apt upgrade  
$ apt install python3-mysqldb # Instalamos la API para acceder a MySQL
```

Integración SQL en Python: Conexión a BBDD

Para conectarnos a la base de datos y ejecutar cualquier consulta, el procedimiento consiste en:

1. Abrir la conexión y crear un puntero
2. Ejecutar la consulta
3. Traer los resultados (si de una selección se trata) o hacer efectiva la escritura (cuando se inserta, actualiza o eliminan datos)
4. Cerrar el puntero y la conexión

Nota

Los resultados de una consulta de selección, se reciben en una tupla, cuyos elementos, son otras tuplas, conteniendo el valor de cada campo seleccionado de la tabla, en el orden que han sido seleccionados.

```
import MySQLdb

DB_HOST = 'localhost'
DB_USER = 'root'
DB_PASS = 'mysqlroot'
DB_NAME = 'a'

def run_query(query=''):
    datos = [DB_HOST, DB_USER, DB_PASS, DB_NAME]

    conn = MySQLdb.connect(*datos) # Conectar a la base de datos
    cursor = conn.cursor()         # Crear un cursor
    cursor.execute(query)          # Ejecutar una consulta

    if query.upper().startswith('SELECT'):
        data = cursor.fetchall()   # Traer los resultados de un select
    else:
        conn.commit()              # Hacer efectiva la escritura de datos
        data = None

    cursor.close()                 # Cerrar el cursor
    conn.close()                  # Cerrar la conexión

    return data
```


Integración SQL en Python: Consulta, Inserción y Borrado Datos

Insertar Datos

```
dato = raw_input("Dato: ")
query = "INSERT INTO b (b2) VALUES ('%s')" % dato
run_query(query)
```

Seleccionar Todos los Registros

```
query = "SELECT b1, b2 FROM b ORDER BY b2 DESC"
result = run_query(query)
print result
```

Seleccionar Sólo Registros Coincidentes

```
criterio = raw_input("Ingrese criterio de búsqueda: ")
query = "SELECT b1, b2 FROM b WHERE b2 = '%s'" % criterio
result = run_query(query)
print result
```

Eliminar Registros

```
criterio = raw_input("Ingrese criterio p7 eliminar coincidencias: ")
query = "DELETE FROM b WHERE b2 = '%s'" % criterio
run_query(query)
```

Actualizar Datos

```
b1 = raw_input("ID: ")
b2 = raw_input("Nuevo valor: ")
query = "UPDATE b SET b2='%s' WHERE b1 = %i" % (b2, int(b1))
run_query(query)
```

Integración SQL en Python: Ejemplo Código Python de Consulta

Edición del Script:

```
root@BBDDSERVERMySQL: /home/rsocas/ScriptsPython
GNU nano 4.8 script1.py
import MySQLdb

DB_HOST = 'localhost'
DB_USER = 'rsocas'
DB_PASS = ' '
DB_NAME = 'university'

def run_query(query=''):
    datos = [DB_HOST, DB_USER, DB_PASS, DB_NAME]

    conn = MySQLdb.connect(*datos) # Conectar a la base de datos
    cursor = conn.cursor()         # Crear un cursor
    cursor.execute(query)          # Ejecutar una consulta

    if query.upper().startswith('SELECT'):
        data = cursor.fetchall()   # Traer los resultados de un select
    else:
        conn.commit()              # Hacer efectiva la escritura de datos
        data = None

    cursor.close()                 # Cerrar el cursor
    conn.close()                  # Cerrar la conexión

    return data

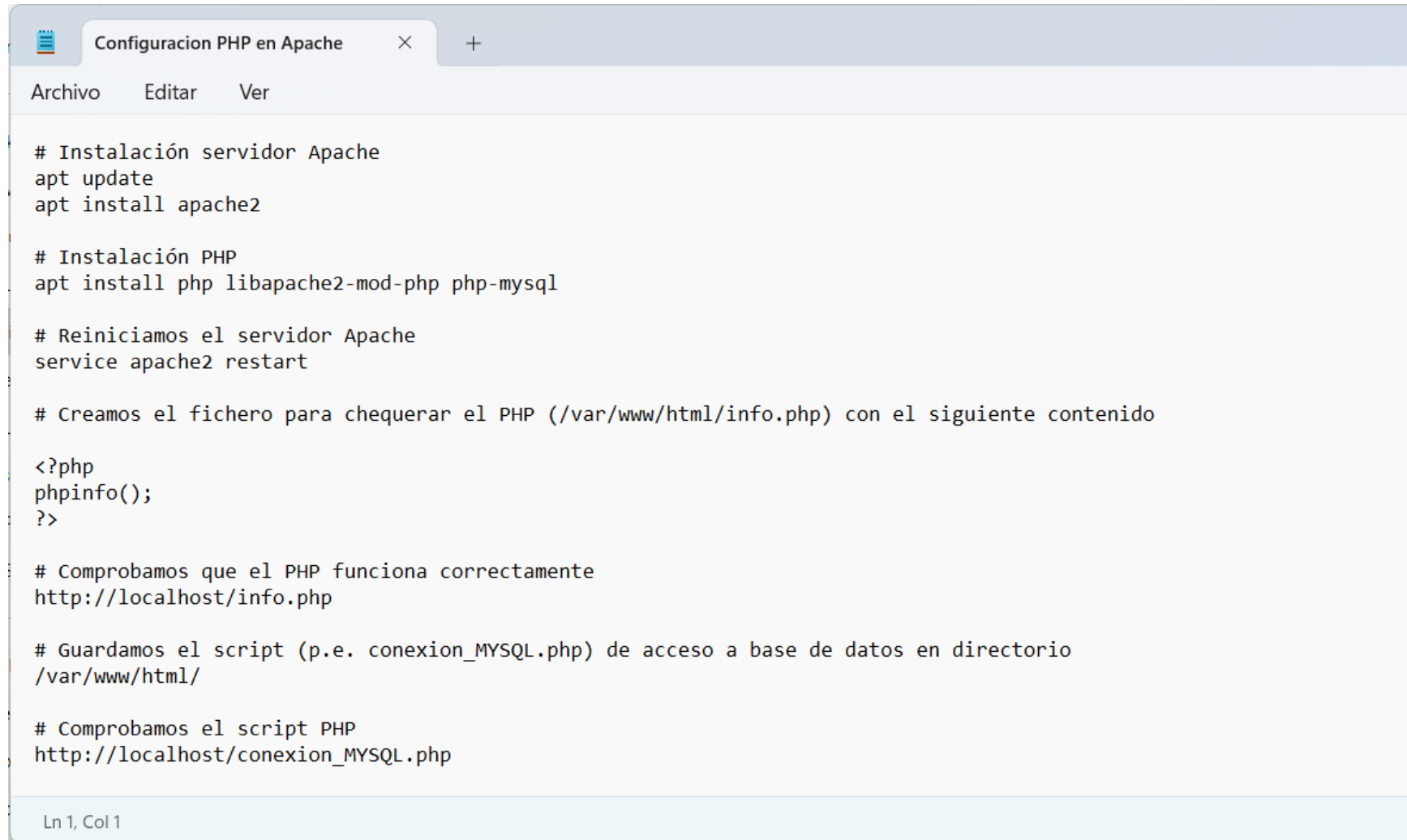
query = "SELECT * FROM course"
result = run_query(query)
print("course_id,title,dept_name,credits")
for row in result:
    course_id=row[0]
    title=row[1]
    dept_name=row[2]
    credits=row[3]
    print("{0},{1},{2},{3}".format(course_id,title,dept_name,credits))

^G Ver ayuda  ^O Guardar  ^W Buscar  ^K Cortar Texto ^J Justificar  ^C Posición  M-U Deshacer
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^T Ortografía ^_ Ir a línea  M-E Rehacer
```

Ejecución del Script:

```
root@BBDDSERVERMySQL: /home/rsocas/ScriptsPython# python3 script1.py
course_id,title,dept_name,credits
BIO-101,Intro. to Biology,Biology,4
BIO-301,Genetics,Biology,4
BIO-399,Computational Biology,Biology,3
CS-101,Intro. to Computer Science,Comp. Sci.,4
CS-190,Game Design,Comp. Sci.,4
CS-315,Robotics,Comp. Sci.,3
CS-319,Image Processing,Comp. Sci.,3
CS-347,Database System Concepts,Comp. Sci.,3
EE-181,Intro. to Digital Systems,Elec. Eng.,3
FIN-201,Investment Banking,Finance,3
HIS-351,World History,History,3
MU-199,Music Video Production,Music,3
PHY-101,Physical Principles,Physics,4
root@BBDDSERVERMySQL: /home/rsocas/ScriptsPython#
```

Configuración PHP en Servidor WEB Apache



```
# Instalación servidor Apache
apt update
apt install apache2

# Instalación PHP
apt install php libapache2-mod-php php-mysql

# Reiniciamos el servidor Apache
service apache2 restart

# Creamos el fichero para chequerar el PHP (/var/www/html/info.php) con el siguiente contenido

<?php
phpinfo();
?>

# Comprobamos que el PHP funciona correctamente
http://localhost/info.php

# Guardamos el script (p.e. conexion_MYSQL.php) de acceso a base de datos en directorio
/var/www/html/

# Comprobamos el script PHP
http://localhost/conexion_MYSQL.php
```

Ln 1, Col 1

Plantilla de Script PHP

```
Conexion_MySQL_plantilla.php
Archivo  Editar  Ver

<?php
// Ejemplo de conexión a base de datos MySQL con PHP.
//
// Ejemplo realizado por Oscar Abad Folgueira: http://www.oscarabadfolgueira.com y https://www.dinapyme.com

// Datos de la base de datos. Sustituir los valores entre "" por los correspondientes de la BBDD sobre la que
// se quiere yrabajar
$usuario = "usuario";
$password = "password";
$servidor = "servidor";
$basedatos = "basedatos";
$tabla = "tabla";

// creación de la conexión a la base de datos con mysql_connect()
$conexion = mysqli_connect( $servidor, $usuario, $password ) or die ("No se ha podido conectar al servidor de Base de datos");

// Selección del a base de datos a utilizar
$db = mysqli_select_db( $conexion, $basedatos ) or die ( "Ups! Pues va a ser que no se ha podido conectar a la base de datos" );

// establecer y realizar consulta. guardamos en variable.
$consulta = "SELECT * FROM $tabla";
$resultado = mysqli_query( $conexion, $consulta ) or die ( "Algo ha ido mal en la consulta a la base de datos");

// Motrar el resultado de los registro de la base de datos. Aqui podemos poner los valores que queramos a Campo1, Campo2, etc.
// pero es buna práctica ponerle nombres similares a los de la base de datos
// Encabezado de la tabla
echo "<table borde='2'>";
echo "<tr>";
echo "<th>Campo1</th>";
echo "<th>Campo2</th>";
echo "<th>Campo3</th>";
echo "<th>Campo4</th>";
echo "</tr>";

// Bucle while que recorre cada registro y muestra cada campo en la tabla. Los valores de Campo1, Campo2, etc. deben coincidir de forma exacta
// con los campos de la BBDD
while ($columna = mysqli_fetch_array( $resultado ))
{
    echo "<tr>";
    echo "<td>" . $columna['Campo1'] . "</td><td>" . $columna['Campo2'] . "</td><td>" . $columna['Campo3'] . "</td><td>" . $columna['Campo4'] . "</td>";
    echo "</tr>";
}

echo "</table>"; // Fin de la tabla

// cerrar conexión de base de datos
mysqli_close( $conexion );

?>
```

Ejemplo de Script PHP

```
GNU nano 4.8 script.php
<?php
// Ejemplo de conexión a base de datos MySQL con PHP.
//
// Ejemplo realizado por Oscar Abad Folgueira: http://www.oscarabadfolgueira.com y https://www.dinapyme.com

// Datos de la base de datos. Sustituir los valores entre "" por los correspondientes de la BBDD sobre la que
// se quiere yrabajar
$usuario = "usuario_1";
$password = "Password1!";
$servidor = "localhost";
$basededatos = "classicmodels";
$tabla = "employees";

// creación de la conexión a la base de datos con mysql_connect()
$conexion = mysqli_connect( $servidor, $usuario, $password ) or die ( "No se ha podido conectar al servidor de Base de datos");

// Selección del a base de datos a utilizar
$db = mysqli_select_db( $conexion, $basededatos ) or die ( "Ups! Pues va a ser que no se ha podido conectar a la base de datos" );

// establecer y realizar consulta. guardamos en variable.
$query = "SELECT lastName, firstName, email, jobTitle FROM $tabla";
$resultado = mysqli_query( $conexion, $query ) or die ( "Algo ha ido mal en la consulta a la base de datos");

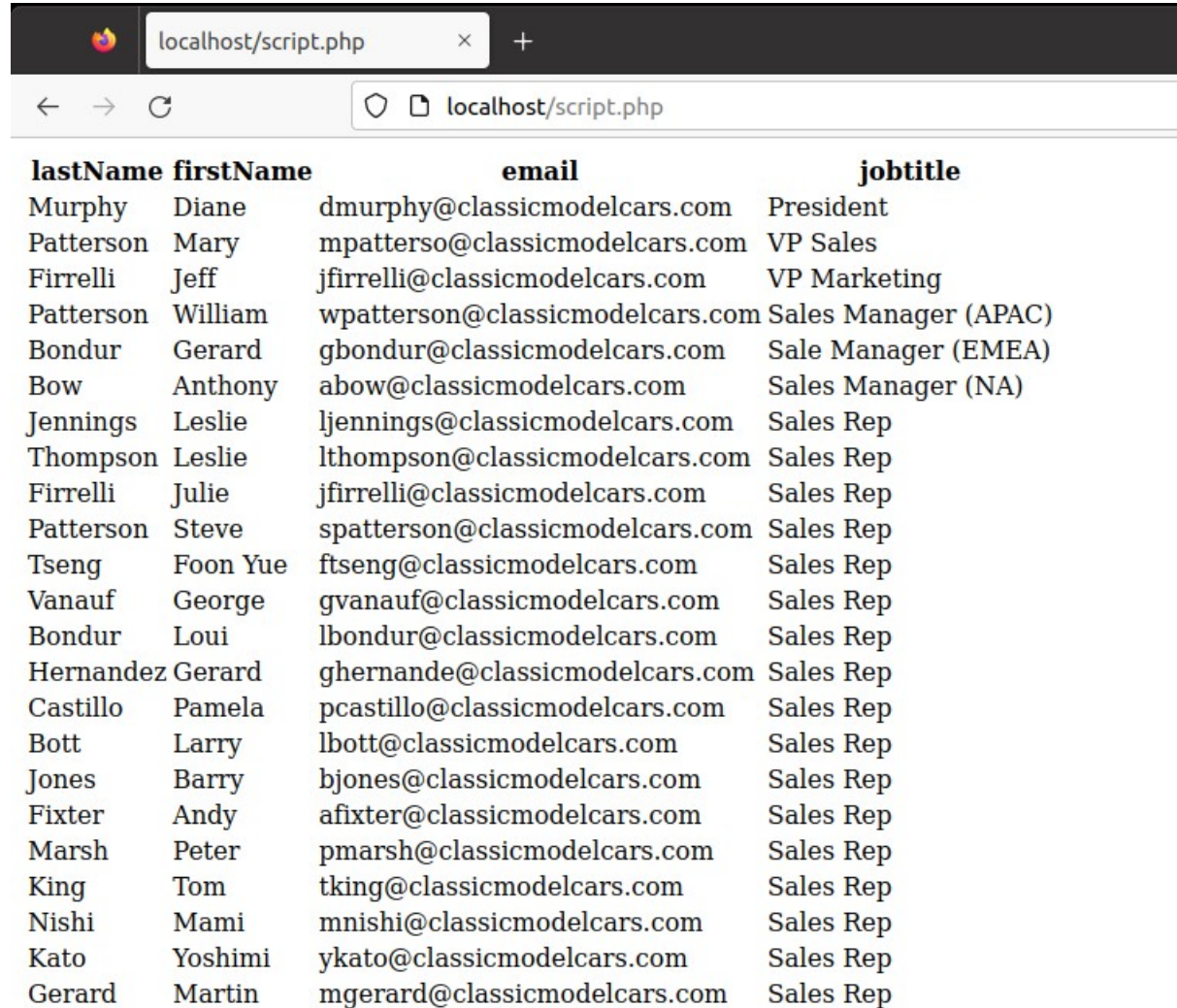
// Motrar el resultado de los registro de la base de datos. Aquí podemos poner los valores que queramos a Campo1, Campo2, etc.
// pero es buna práctica ponerle nombres similares a los de la base de datos
// Encabezado de la tabla
echo "<table borde='2'>";
echo "<tr>";
echo "<th>lastName</th>";
echo "<th>firstName</th>";
echo "<th>email</th>";
echo "<th>jobTitle</th>";
echo "</tr>";

// Bucle while que recorre cada registro y muestra cada campo en la tabla. Los valores de Campo1, Campo2, etc. deben coincidir de forma exacta
// con los campos de la BBDD
while ( $columna = mysqli_fetch_array( $resultado ) )
{
    echo "<tr>";
    echo "<td>" . $columna['lastName'] . "</td><td>" . $columna['firstName'] . "</td><td>" . $columna['email'] . "</td><td>" . $columna['jobTitle'] . "</td>";
    echo "</tr>";
}

echo "</table>"; // Fin de la tabla

// cerrar conexión de base de datos
mysqli_close( $conexion );
?>
```


Ejemplo de Ejecución PHP



The image shows a web browser window with the address bar displaying 'localhost/script.php'. The page content is a table with four columns: 'lastName', 'firstName', 'email', and 'jobtitle'. The table lists 20 employees of Classic Model Cars, including their names, email addresses, and job titles.

lastName	firstName	email	jobtitle
Murphy	Diane	dmurphy@classicmodelcars.com	President
Patterson	Mary	mpatterso@classicmodelcars.com	VP Sales
Firrelli	Jeff	jfirrelli@classicmodelcars.com	VP Marketing
Patterson	William	wpatterson@classicmodelcars.com	Sales Manager (APAC)
Bondur	Gerard	gbondur@classicmodelcars.com	Sale Manager (EMEA)
Bow	Anthony	abow@classicmodelcars.com	Sales Manager (NA)
Jennings	Leslie	ljennings@classicmodelcars.com	Sales Rep
Thompson	Leslie	lthompson@classicmodelcars.com	Sales Rep
Firrelli	Julie	jfirrelli@classicmodelcars.com	Sales Rep
Patterson	Steve	spatterson@classicmodelcars.com	Sales Rep
Tseng	Foon Yue	ftseng@classicmodelcars.com	Sales Rep
Vanauf	George	gvanauf@classicmodelcars.com	Sales Rep
Bondur	Loui	lbondur@classicmodelcars.com	Sales Rep
Hernandez	Gerard	ghernande@classicmodelcars.com	Sales Rep
Castillo	Pamela	pcastillo@classicmodelcars.com	Sales Rep
Bott	Larry	lbott@classicmodelcars.com	Sales Rep
Jones	Barry	bjones@classicmodelcars.com	Sales Rep
Fixter	Andy	afixter@classicmodelcars.com	Sales Rep
Marsh	Peter	pmarsh@classicmodelcars.com	Sales Rep
King	Tom	tking@classicmodelcars.com	Sales Rep
Nishi	Mami	mnishi@classicmodelcars.com	Sales Rep
Kato	Yoshimi	ykato@classicmodelcars.com	Sales Rep
Gerard	Martin	mgerard@classicmodelcars.com	Sales Rep

5. Tipos de Almacenamiento

Tipos de Características del Almacenamiento

El almacenamiento es la capacidad que tienen los sistemas electrónicos para guardar información. Para los sistemas de bases de datos (DBMS) es un elemento clave para que puedan realizar sus funciones.

Desde el punto de vista de la persistencia se puede diferenciar en:

- **Almacenamiento volátil:** Se pierde contenido cuando falta la alimentación, normalmente el suministro eléctrico.
- **Almacenamiento no volátil:** El contenido persiste incluso cuando se apaga el sistema. También se incluye aquí el almacenamiento secundario y terciario, así como los sistemas cuya memoria principal está respaldada por baterías.

Por otro lado, desde el punto de vista del performance, los parámetros más relevantes que los definen son:

- **Velocidad de acceso:** Cuánto tiempo tarda el sistema en leer o escribir la información. Se suele medir en MB/s (megabytes por segundo).
- **Coste de almacenamiento:** Es el coste de almacenar la información, normalmente se expresa en €/GB (euros por gigabyte).
- **Fiabilidad:** La fiabilidad se mide mediante el (MTBF) *Mean Time Between Failures* o tiempo medio entre fallos que normalmente se mide en millones de horas.

5. Tipos de Almacenamiento

Dispositivos de Almacenamiento

Memorias

- **Cache:** Es la forma de almacenamiento más rápida, pero también la más cara. Normalmente tiene poco espacio de almacenamiento y la gestiona el sistema operativo y no el DBMS. Se trata de una memoria volátil.
- **Memoria principal:** Es la memoria que utiliza el sistema para todas sus operaciones. Es mayor que la cache, pero no lo suficiente como para contener toda la DB, o si la usáramos para esto, sería demasiada cara. Es también volátil y los datos se pierden en caso de falta de energía.
- **Memoria flash:** Se trata de una memoria no volátil y con tiempos de lecturas similares a la memoria principal. La principal desventaja respecto a la memoria principal es que los tiempos de escritura son muchos más largos. Hoy en día tienen capacidades suficientes como para almacenar toda la DB en ella.

Discos

- **Discos magnéticos:** Ha sido por muchos años el medio de almacenamiento por excelencia para las DB. Es una memoria no volátil y dispone de espacio suficiente para albergar toda la información. Es relativamente barata, pero tiene una velocidad de acceso inferior a las anteriores. Téngase en cuenta que, para actuar sobre los datos hay que llevarlos a memoria, hacer las operaciones necesarias y volverlos a guardar en el disco nuevamente. Esta operación se realiza con los otros sistemas de almacenamiento en memoria también, pero en este caso, la latencia es muy superior.
- **Discos ópticos:** Tienen mucha más capacidad que los discos magnéticos, pero por contra, ofrece peores prestaciones de lectura/escritura. Hay muchos tipos, desde los que se permite una sola escritura (ROM Read Only Memory), hasta los que permiten varias de ellas (RW Read and Write). Como no podría ser de otra forma, son sistemas más económicos que todos los anteriores.
- **Cintas magnéticas:** Su principal uso es realizar copias de seguridad, es mucho más barata que los discos, pero el acceso es más lento, ya que, hay que acceder de forma secuencial (desde comienzo) y no de forma directa como se hace con los discos.

5. Tipos de Almacenamiento

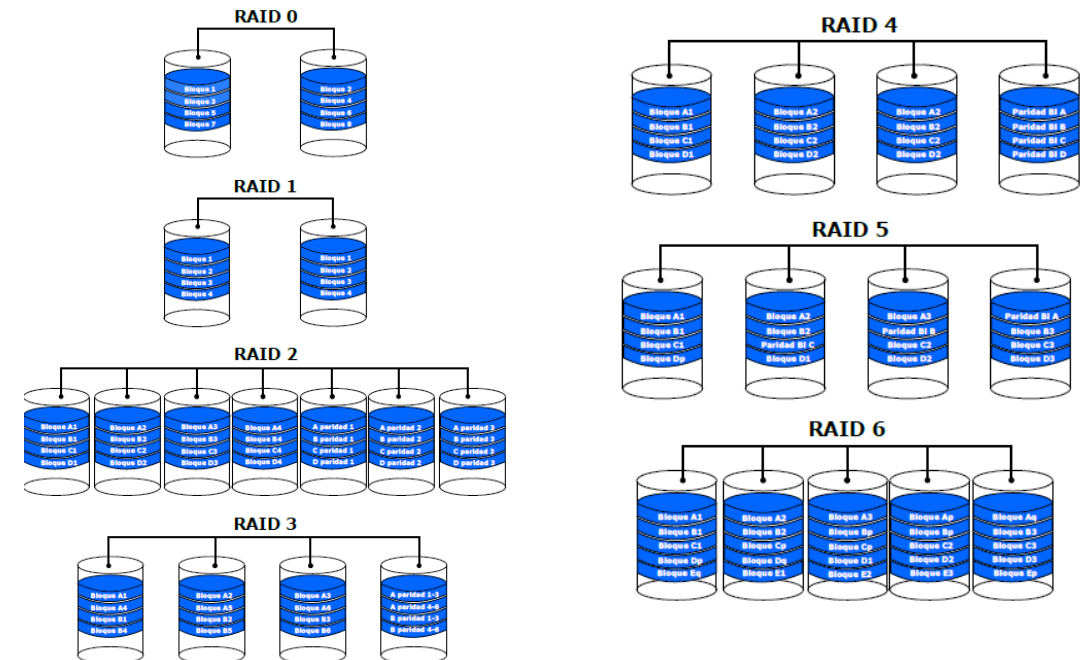
Almacenamiento Redundante RAID

Las técnicas de organización de discos denominadas matrices redundantes de discos independientes (RAID Redundant Arrays of Independent Disks) se usan para lograr un mejor rendimiento y fiabilidad de los sistemas de almacenamiento. Por tanto, la filosofía del RAID se basa en:

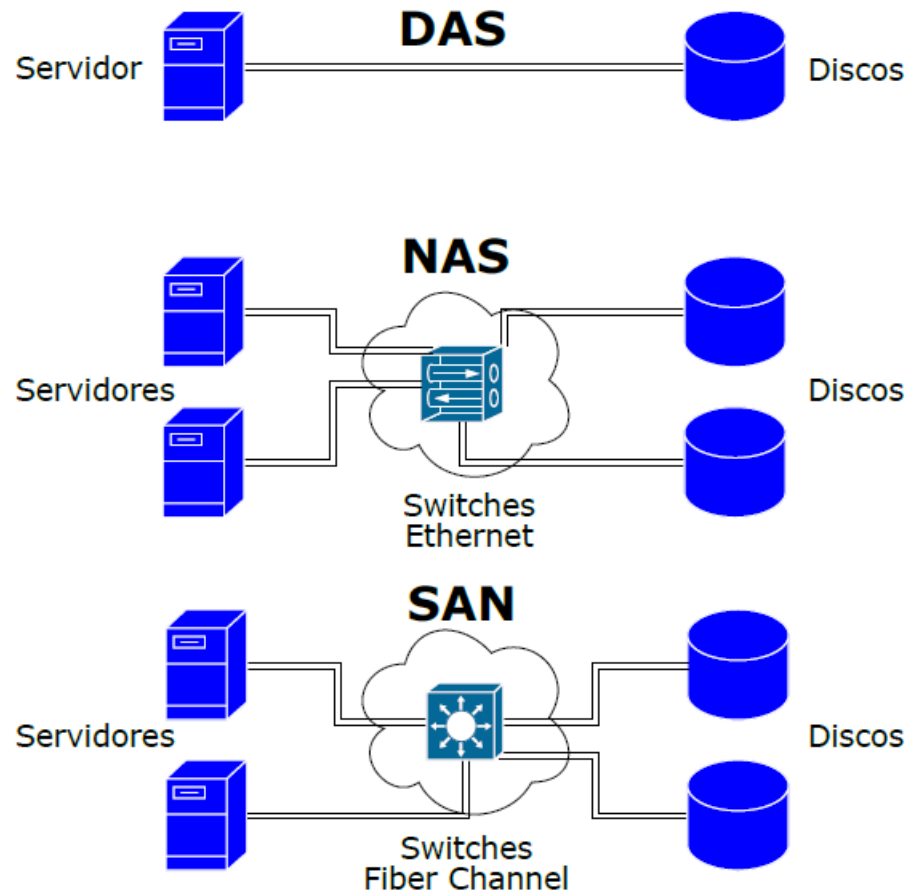
Características

- **Fiabilidad:** Se incrementa aplicando redundancia. El planteamiento más sencillo para introducir redundancia en el sistema es duplicar cada disco.
- **Prestaciones:** Se distribuyen los datos de manera equitativa entre las unidades que lo forman. Esta distribución equitativa de los datos hace que, prácticamente, se duplique la velocidad de acceso dado que cada disco transmite simultáneamente los datos de forma paralela.

Arquitecturas Típicas



Sistemas de Almacenamiento (1/2)



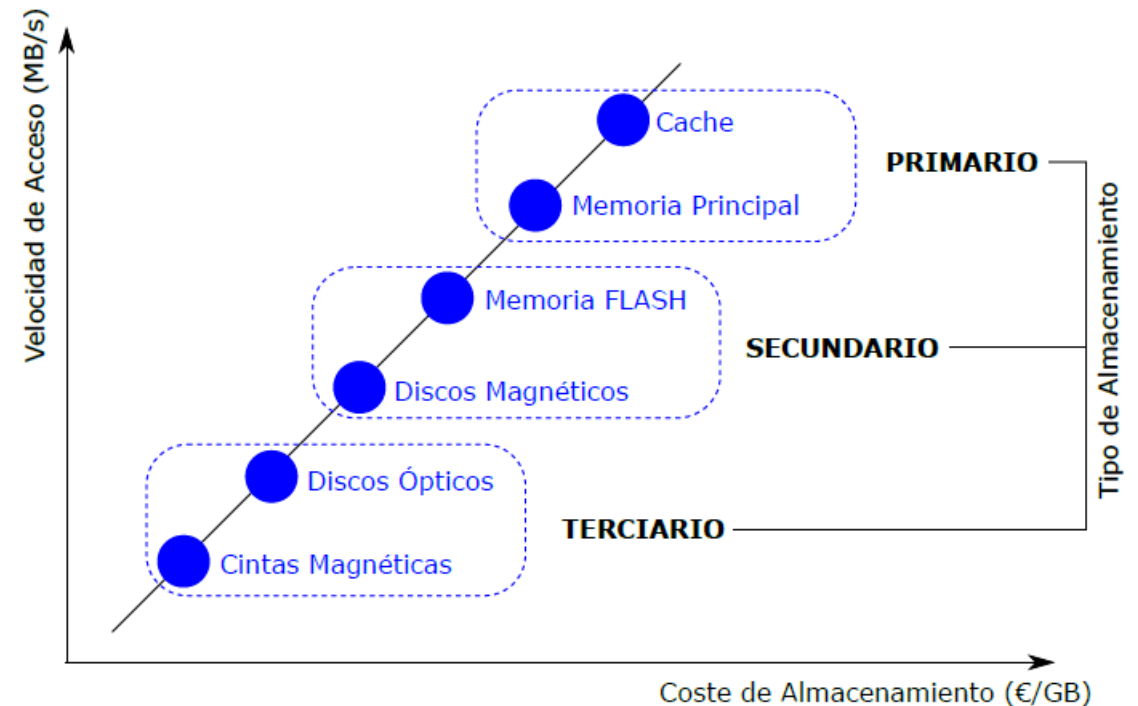
- **Sistema DAS (Directly Attached Storage):** El concepto es muy sencillo y significa que el sistema de almacenamiento está directamente conectado a servidor a través de un cable. Este sistema de almacenamiento puede ser desde un único disco hasta un array de múltiples discos en una configuración RAID.
- **Sistema NAS (Network Attached Storage):** Este sistema tiene la función de trabajar como un disco duro conectado directamente a una red de área local (LAN). Con estos discos en red, los usuarios pueden compartir archivos y trabajar sobre ellos.
- **Sistema SAN (Storage Area Network):** Se trata de una red de datos de alta velocidad (Fiber Channel) que proporciona acceso al almacenamiento a nivel de bloque. Su funcionalidad hace que un conjunto de dispositivos de almacenamiento sea accesible para múltiples servidores.

Sistemas de Almacenamiento (2/2)

	DAS	NAS	SAN
Protocols	SATA, SAS	SMB (CIFS), NFS	iSCSI, Fiber Channel
Type	File Storage	Block Storage	Block Storage
Speed	5-10ms	5-10ms	20-50ms
Data Transmission	IDE/SCSI	Ethernet, TCP/IP	FiberChannel
Complexity	Easy	Moderate	Complex
Management Costs	High	Moderate	Low
Storage Type	Sectors	Shared Files	Blocks
Supports Capacity Sharing?	No (possible manually)	Yes	Yes
Connected to Network or not?	No	Yes	Yes
Scalability	Low	Medium	High

Por otro lado, los dispositivos de almacenamiento los podemos agrupar según la siguiente jerarquía:

- **Almacenamiento primario:** Es el medio más rápido, pero es volátil. En este grupo estaría la memoria cache y la memoria principal.
- **Almacenamiento secundario:** También se denomina almacenamiento on-line, es un almacenamiento no volátil y con tiempo de acceso moderado. En esta categoría están la memoria flash y los discos magnéticos.
- **Almacenamiento terciario:** Es también un almacenamiento no volátil, pero de baja velocidad de acceso. Se denomina también almacenamiento off-line y se usa normalmente para guardar copias de seguridad.





Índices: Concepto

MySQL uses indexes to quickly find rows with specific column values. Without an index, MySQL must scan the whole table to locate the relevant rows. The larger table, the slower it searches.

In this section, you will learn about MySQL index including creating indexes, removing indexes, listing all indexes of a table and other important features of indexes in MySQL.

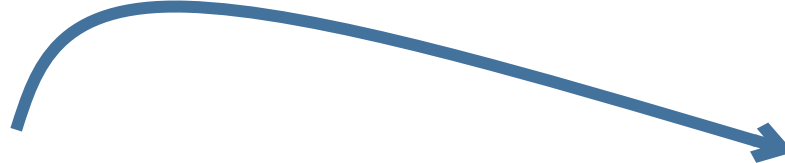
An index is a data structure such as B-Tree that improves the speed of data retrieval on a table at the cost of additional writes and storage to maintain it. The query optimizer may use indexes to quickly locate data without having to scan every row in a table for a given query.

When you create a table with a primary key or unique key, MySQL automatically creates a special index named PRIMARY. This index is called the clustered index. The PRIMARY index is special because the index itself is stored together with the data in the same table. The clustered index enforces the order of rows in the table.

Other indexes than the PRIMARY index are called secondary indexes or non-clustered indexes.

Crear un Índice

```
SELECT
    employeeNumber,
    lastName,
    firstName
FROM
    employees
WHERE
    jobTitle = 'Sales Rep';
```



```
CREATE INDEX jobTitle ON employees(jobTitle);
```

```
EXPLAIN SELECT
    employeeNumber,
    lastName,
    firstName
FROM
    employees
WHERE
    jobTitle = 'Sales Rep';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	employees	NULL	ALL	NULL	NULL	NULL	NULL	23	10.00	Using where

As you can see, MySQL had to scan the whole table which consists of 23 rows to find the employees with the Sales Rep job title.

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	employees	NULL	ref	jobTitle	jobTitle	52	const	17	100.00	NULL

As you can see, MySQL just had to locate 17 rows from the jobTitle index as indicated in the key column without scanning the whole table.

7. Indexado



Eliminar un Índice

```
CREATE TABLE leads(  
    lead_id INT AUTO_INCREMENT,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) NOT NULL,  
    information_source VARCHAR(255),  
    INDEX name(first_name,last_name),  
    UNIQUE email(email),  
    PRIMARY KEY(lead_id)  
);
```

The following statement drops the email index from the leads table with a specific algorithm and lock:

```
DROP INDEX name ON leads;
```


Mostrar los Índices de una tabla

```
CREATE TABLE contacts(
    contact_id INT AUTO_INCREMENT,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    PRIMARY KEY(contact_id),
    UNIQUE(email),
    INDEX phone(phone) INVISIBLE,
    INDEX name(first_name, last_name) comment 'By first name and/or last name'
);
```

```
SHOW INDEXES FROM contacts;
```

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible
►	contacts	0	PRIMARY	1	contact_id	A	0	NULL	NULL		BTREE			YES
	contacts	0	email	1	email	A	0	NULL	NULL	YES	BTREE			YES
	contacts	1	phone	1	phone	A	0	NULL	NULL	YES	BTREE			NO
	contacts	1	name	1	first_name	A	0	NULL	NULL		BTREE	By first name and/or last name		YES
	contacts	1	name	2	last_name	A	0	NULL	NULL		BTREE	By first name and/or last name		YES

- **Table:** The name of the table
- **Non_unique:** 1 if the index can contain duplicates, 0 if it cannot.
- **Key_name:** The name of the index. The primary key index always has the name of PRIMARY.
- **Seq_in_index:** The column sequence number in the index. The first column sequence number starts from 1.
- **Column_name:** The column name
- **Collation:** Collation represents how the column is sorted in the index. A means ascending, B means descending, or NULL means not sorted.
- **Cardinality:** The cardinality returns an estimated number of unique values in the index. Note that the higher the cardinality, the greater the chance that the query optimizer uses the index for lookups.
- **Sub_part:** The index prefix. It is null if the entire column is indexed. Otherwise, it shows the number of indexed characters in case the column is partially indexed.
- **Packed:** indicates how the key is packed; NUL if it is not.
- **Null:** YES if the column may contain NULL values and blank if it does not.
- **Index_type:** represents the index method used such as BTREE, HASH, RTREE, or FULLTEXT.
- **Comment:** The information about the index not described in its own column.
- **Index_comment:** shows the comment for the index specified when you create the index with the COMMENT attribute.
- **Visible:** Whether the index is visible or invisible to the query optimizer or not; YES if it is, NO if not.
- **Expression:** If the index uses an expression rather than column or column prefix value, the expression indicates the expression for the key part and also the column_name column is NULL.



 Calle Playa de Liencres, 2 bis
(entrada por calle Rozabella)
Parque Europa Empresarial
Edificio Madrid
28290 Las Rozas, Madrid

 900 373 379  info@u-tad.com

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

